

Decision Tree

Random Forest

Decision Tree Example

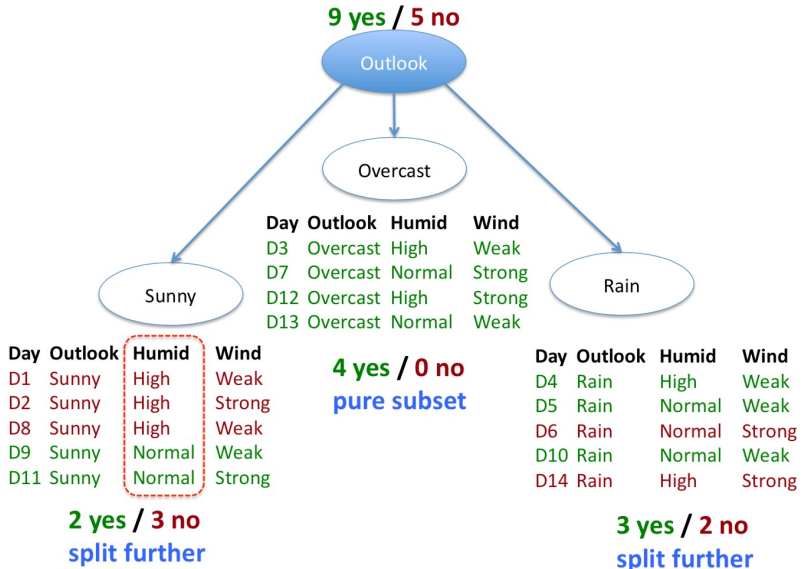
Training examples: 9 yes / 5 no

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

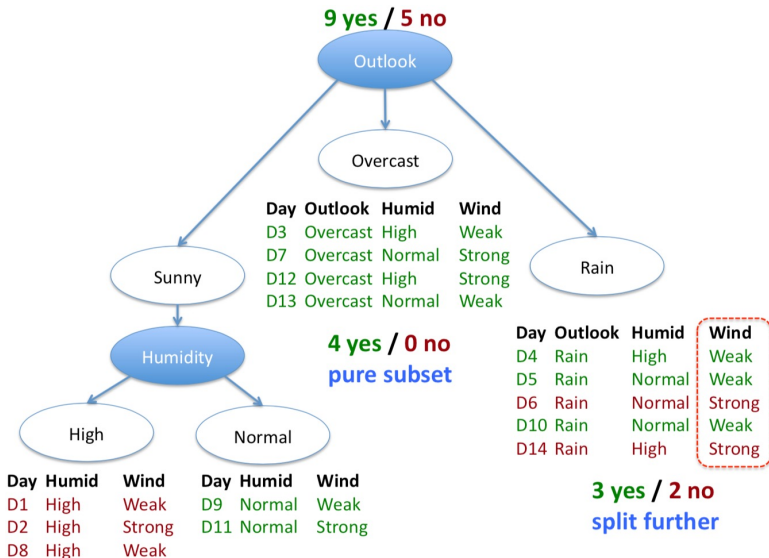
New data:

D15 Rain High Weak ?

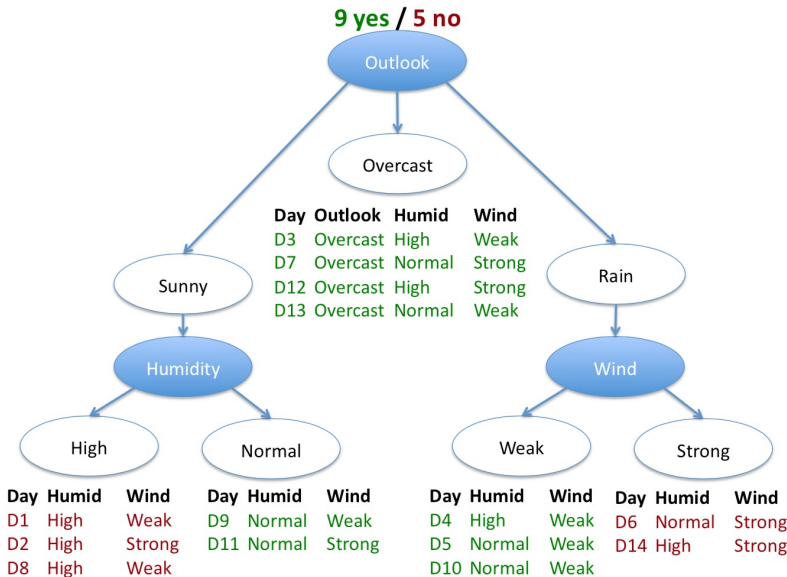
Decision Tree Example



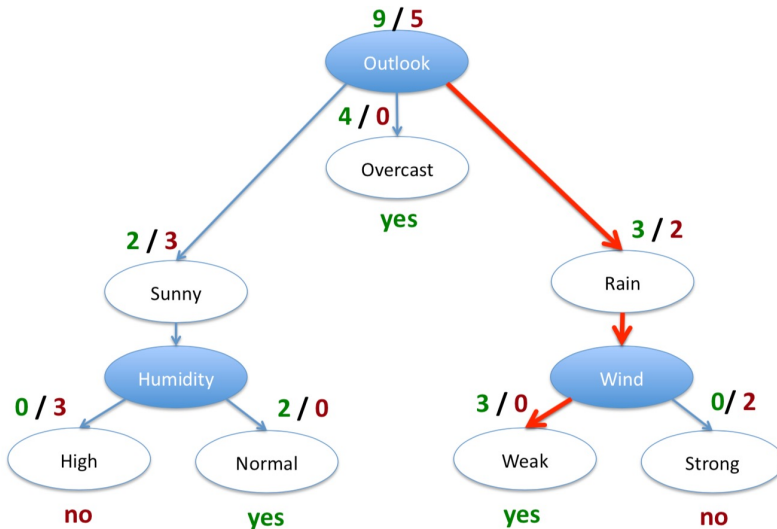
Decision Tree Example



Decision Tree Example



Decision Tree Example



New data:

Day	Outlook	Humid	Wind
D15	Rain	High	Weak

→ Yes

ID3 Algorithm

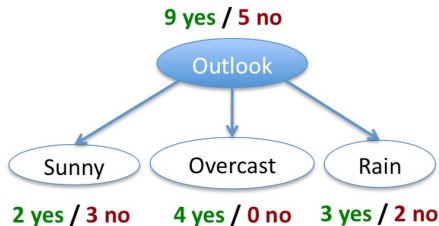
node = Root

examples = Training Set

Split (**node**, {**examples**}):

1. Find A, the **best attribute** for splitting the {**examples**}
2. Create decision nodes for attribute A (i.e., **child nodes** of **node**)
3. Split training {**examples**} to **child nodes**
4. If examples perfectly classified (subset is pure): STOP
else: iterate over new child nodes
 Split (**child_node**, {**subset of examples**})

Find The Best Attribute



- Want to measure “purity” of the split
 - more certain about Yes/No after the split
 - pure set (4 yes / 0 no) => completely certain (100%)
 - impure (3 yes / 3 no) => completely uncertain (50%)
 - can’t use $P(\text{“yes”} \mid \text{set})$:
 - must be symmetric: 4 yes / 0 no as pure as 0 yes / 4 no

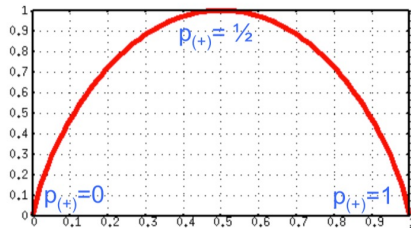
Entropy

- Entropy: $H(S) = - p_{(+)} \log_2 p_{(+)} - p_{(-)} \log_2 p_{(-)}$ bits
 - S ... subset of training examples
 - $p_{(+)} / p_{(-)}$... % of positive / negative examples in S
- Interpretation: assume item X belongs to S
 - how many bits need to tell if X positive or negative
- impure (3 yes / 3 no):

$$H(S) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1 \text{ bits}$$

- pure set (4 yes / 0 no):

$$H(S) = -\frac{4}{4} \log_2 \frac{4}{4} - \frac{0}{4} \log_2 \frac{0}{4} = 0 \text{ bits}$$



Information Gain

- Want many items in pure sets
- Expected drop in entropy after split:

$$\text{Gain}(S, A) = H(S) - \sum_{V \in \text{Values}(A)} \frac{|S_V|}{|S|} H(S_V)$$

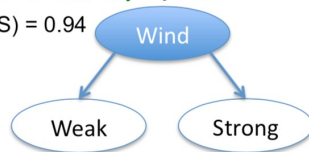
V ... possible values of A
 S ... set of examples $\{X\}$
 S_V ... subset where $X_A = V$

- Mutual Information
 - between attribute A and class labels of S

$$\begin{aligned}\text{Gain}(S, \text{Wind}) &= H(S) - \frac{8}{14} H(S_{\text{weak}}) - \frac{6}{14} H(S_{\text{strong}}) \\ &= 0.94 - \frac{8}{14} * 0.81 - \frac{6}{14} * 1.0 \\ &= 0.049\end{aligned}$$

$$-\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \quad \text{9 yes / 5 no}$$

$$H(S) = 0.94$$



6 yes / 2 no

$$-\frac{6}{8} \log_2 \frac{6}{8} - \frac{2}{8} \log_2 \frac{2}{8}$$

$$H(S_{\text{weak}}) = 0.81$$

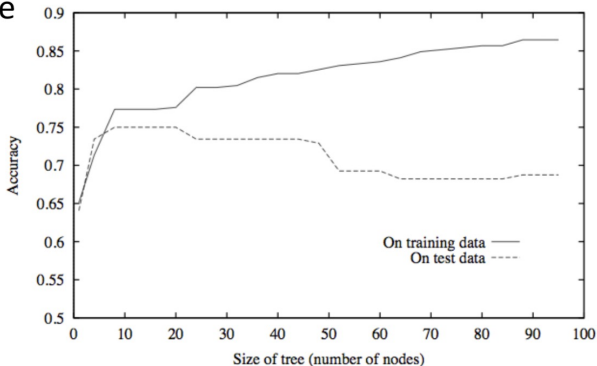
3 yes / 3 no

$$-\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6}$$

$$H(S_{\text{strong}}) = 1.0$$

Overfitting in Decision Tree

- Can always classify training examples perfectly
 - keep splitting until each node contains 1 example
 - singleton = pure
- Doesn't work on new data



Avoid Overfitting

How can we avoid overfitting?

- Stop growing when data split is not statistically significant.
- Acquire more training data.
- Remove irrelevant attributes (manual process – not always possible)
- Grow full tree, then post-prune.

How to select “best” tree:

- Measure performance over training data
- Measure performance over separate validation data set
- Add complexity penalty to performance measure

Pre-pruning that stop growing the tree earlier, before it perfectly classifies the training set.

Post-pruning that allows the tree to perfectly classify the training set, and then post prune the tree.

General Structure

Task: classification, discriminative

- Model structure: decision tree
 - Score function
 - Information gain at each node
 - Preference for short trees
 - Preference for high-gain attributes near the root
- Optimization / search method
 - Greedy search from simple to complex
 - Guided by information gain

Problem With Information Gain

- Biased towards attributes with many values

- Won't work for new data: D15 Rain High Weak

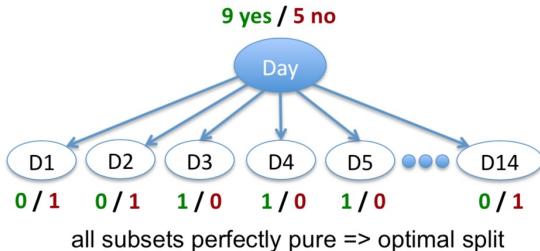
- Use GainRatio:

$$SplitEntropy(S,A) = - \sum_{V \in Values(A)} \frac{|S_V|}{|S|} \log \frac{|S_V|}{|S|}$$

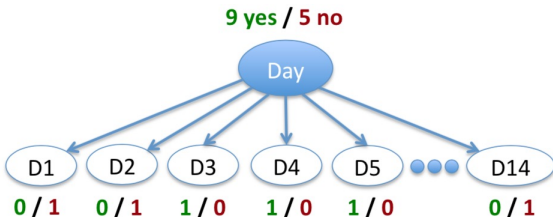
A ... candidate attribute
V ... possible values of A
S ... set of examples {X}
S_V ... subset where X_A = V

$$GainRatio(S,A) = \frac{Gain(S,A)}{SplitEntropy(S,A)}$$

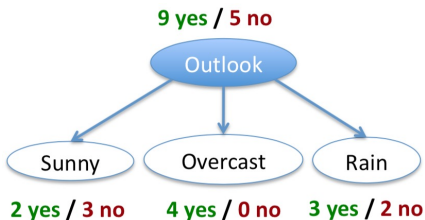
penalizes attributes
with many values



Problem With Information Gain



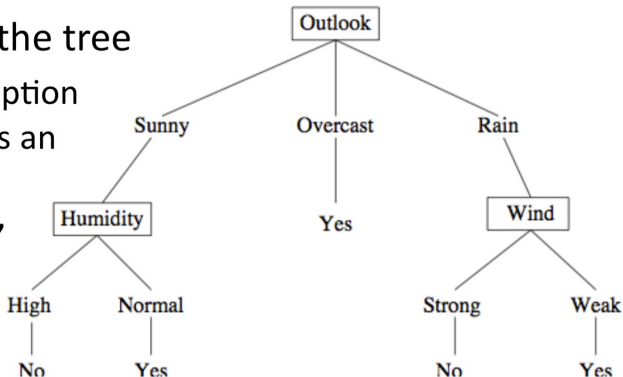
$$\begin{aligned} \text{SplitEntropy}(S, \text{Day}) \\ = -14 \left(\frac{1}{14} \log \frac{1}{14} \right) = 3.80 \end{aligned}$$



$$\begin{aligned} \text{SplitEntropy}(S, \text{Outlook}) \\ = -\frac{5}{14} \log \frac{5}{14} - \frac{4}{14} \log \frac{4}{14} - \frac{5}{14} \log \frac{5}{14} = 1.57 \end{aligned}$$

Interpretation of Trees

- Read rules off the tree
 - concise description of what makes an item positive
- No “black box”
 - important for users

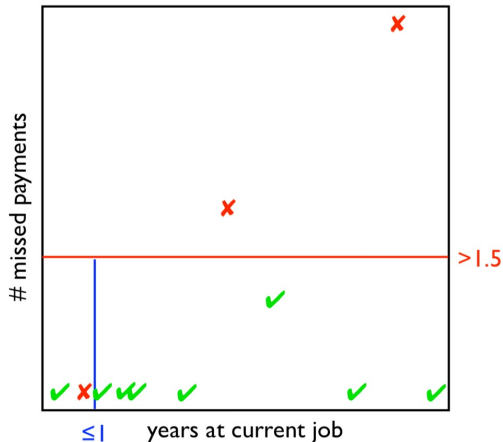


Rule: $(\text{Outlook} = \text{Overcast}) \vee$
 $(\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak}) \vee$
 $(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal})$

Continuous Variables

Predicting credit risk

years at current job	# missed payments	defaulted?
7	0	N
0.75	0	Y
3	0	N
9	0	N
4	2	Y
0.25	0	N
5	1	N
8	4	Y
1.0	0	N
1.75	0	N



Continuous Variables

Outlook	Temp	Humidity	Windy	Play
Sunny	85	85	False	No
Sunny	80	90	True	No
Overcast	83	78	False	Yes
Rainy	70	96	False	Yes
Rainy	68	80	False	Yes
Rainy	65	70	True	No
Overcast	64	65	True	Yes
Sunny	72	95	False	No
Sunny	69	70	False	Yes
Rainy	75	80	False	Yes
Sunny	75	70	True	Yes
Overcast	72	90	True	Yes
Overcast	81	75	False	Yes
Rainy	71	80	True	No

Continuous Temperature Attribute

- First, sort the temperature values, including the class labels
- Then, check all the cut points and choose the one with the best information gain.

64	65	68	69	70	71	72	72	75	75	80	81	83	85
Yes	No	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	Yes	Yes	No

- $Temp < 71.5$: yes = 4, no = 2
- $Temp \geq 71.5$: yes = 5, no = 3

How to compute:

- $H(Temp < 71.5) = ?$
- $H(Temp \geq 71.5) = ?$
- $H(S) = ?$
- $Gain(S, Temp) = ?$

Continuous Temperature Attribute

64	65	68	69	70	71	72	72	75	75	80	81	83	85
Yes	No	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	Yes	Yes	No

Temp < 71.5: yes = 4, no = 2

Temp ≥ 71.5: yes = 5, no = 3

$$H(\text{Temp} < 71.5) = -\frac{4}{6}\log\frac{4}{6} - \frac{2}{6}\log\frac{2}{6} = 0.918$$

$$H(\text{Temp} \geq 71.5) = -\frac{5}{8}\log\frac{5}{8} - \frac{3}{8}\log\frac{3}{8} = 0.954$$

$$H(S) = -\frac{9}{14}\log\frac{9}{14} - \frac{5}{14}\log\frac{5}{14} = 0.940$$

$$\begin{aligned} \text{Gain}(S, \text{Temp}) &= H(S) - \frac{6}{14}H(\text{Temp} < 71.5) - \frac{8}{14}H(\text{Temp} \geq 71.5) \\ &= 0.940 - \frac{6}{14} * 0.918 - \frac{8}{14} * 0.954 = 0.001 \end{aligned}$$

Continuous Temperature Attribute

- First, sort the temperature values, including the class labels
- Then, check all the cut points and choose the one with the best information gain.

64	65	68	69	70	71	72	72	75	75	80	81	83	85
Yes	No	Yes	Yes	Yes	No	No	Yes	Yes	Yes	No	Yes	Yes	No

- $\text{Temp} < 71.5$: yes = 4, no = 2
- $\text{Temp} \geq 71.5$: yes = 5, no = 3
- Place split points **halfway** between values.
- Able to evaluate all split points in **one pass**.

Information Gain for Humidity

Humidity	Play
65	Yes
70	No
70	Yes
70	Yes
75	Yes
78	Yes
80	Yes
80	Yes
80	No
85	No
90	No
90	Yes
95	No
96	Yes

sort the
attribute
values



Humidity	Play
65	Yes
70	No
70	Yes
70	Yes
75	Yes
78	Yes
80	Yes
80	Yes
80	No
85	No
90	No
90	Yes
95	No
96	Yes

compute the gain for
every possible split

what is the information
gain if we split here?

Information Gain for Humidity

Humidity	Play	# of Yes	% of Yes	# of No	% of No	Weight	Entropy Left	# of Yes	% of Yes	# of No	% of No	Weight	Entropy Right	Information Gain
65	Yes	1	100.00%	0	0.00%	7.14%	0.00	8.00	0.62	5.00	0.38	92.86%	0.96	0.0477
70	No	1	50.00%	1	50.00%	14.29%	1.00	8.00	0.67	4.00	0.33	85.71%	0.92	0.0103
70	Yes	2	66.67%	1	33.33%	21.43%	0.92	7.00	0.64	4.00	0.36	78.57%	0.95	0.0005
70	Yes	3	75.00%	1	25.00%	28.57%	0.81	6.00	0.60	4.00	0.40	71.43%	0.97	0.0150
75	Yes	4	80.00%	1	20.00%	35.71%	0.72	5.00	0.56	4.00	0.44	64.29%	0.99	0.0453
78	Yes	5	83.33%	1	16.67%	42.86%	0.65	4.00	0.50	4.00	0.50	57.14%	1.00	0.0903
80	Yes	6	85.71%	1	14.29%	50.00%	0.59	3.00	0.43	4.00	0.57	50.00%	0.99	0.1518
80	Yes	7	87.50%	1	12.50%	57.14%	0.54	2.00	0.33	4.00	0.67	42.86%	0.92	0.2361
80	No	7	77.78%	2	22.22%	64.29%	0.76	2.00	0.40	3.00	0.60	35.71%	0.97	0.1022
85	No	7	70.00%	3	30.00%	71.43%	0.88	2.00	0.50	2.00	0.50	28.57%	1.00	0.0251
90	No	7	63.64%	4	36.36%	78.57%	0.95	2.00	0.67	1.00	0.33	21.43%	0.92	0.0005
90	Yes	8	66.67%	4	33.33%	85.71%	0.92	1.00	0.50	1.00	0.50	14.29%	1.00	0.0103
95	No	8	61.54%	5	38.46%	92.86%	0.96	1.00	1.00	0.00	0.00	7.14%	0.00	0.0477
96	Yes	9	64.29%	5	35.71%	100.00%	0.94	0.00	0.00	0.00	0.00	0.00%	0.00	0.0000

Left branch, e.g., < 67.5

Right branch, e.g., ≥ 67.5

Multi-class Classification

Multiclass classification:

- Predict the **best attribute** to split on.
- Entropy:

$$H(S) = - \sum_c p_c \log_2(p_c).$$

- p_c : % of examples of class **c** in S .

Random (Decision) Forest

Training: grow **K** different decision trees:

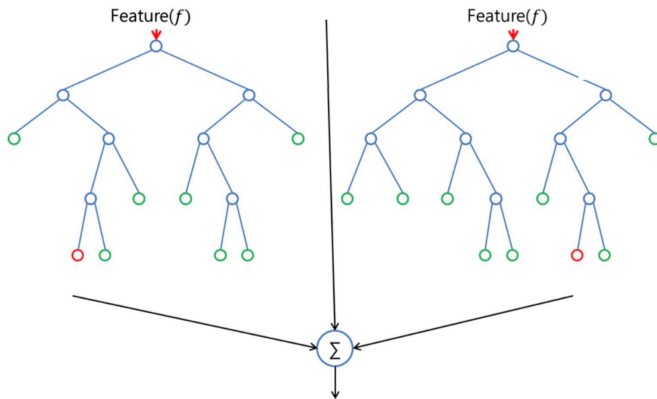
- Pick a **random subset** S_{random} of training examples.
- Grow a full decision tree (no pruning), compute information gain based on S_{random} instead of full set.
- Repeat for **K decision trees**.

Inference: given a new data point X:

- Classify X using each of the K trees.
- Use majority vote: class predicted most often.

Fast, scalable, state-of-the-art performance.

Random (Decision) Forest



Random forest with majority voting

Summary

- ID3: grows decision tree from the root down
 - Greedily selects next best attribute (gain).
- Searches a complete hypothesis space
 - Prefers smaller trees, high gain at the root.
- Overfitting addressed by pruning
 - Prune nodes, while accuracy improves on validation set.
- Can handle missing data.
 - Don't need to go through the whole tree.
- Easily handles irrelevant variables
 - Information Gain = 0 => will not be selected.

